

四旋翼无人机三轴姿态的智能控制

一、选题与系统建模

1. 选题背景

四旋翼无人机因其垂直起降灵活性和机动性能，广泛应用于测绘、巡检、物流等领域，而其姿态控制直接关系到飞行稳定性与任务精度。传统PID控制器虽然结构简单、易于实现，但在多轴耦合、外部扰动和非线性动态作用下容易出现超调、振荡和稳态误差；模糊控制虽能增强鲁棒性，但在面对复杂扰动时仍存在响应不稳定的问题。结合现代智能控制理论，利用神经网络或自适应模糊控制方法能够在线学习系统非线性特性，实现更高精度、更快响应和更强鲁棒性的三轴姿态控制。

因此，本作业选取“四旋翼无人机三轴姿态智能控制”作为研究对象，旨在通过对比传统PID、模糊控制与神经网络控制的性能，探索适用于实际飞行环境的高效智能控制策略，同时深化对无人机动力学建模与控制方法的理解。

2. 系统建模

基于四旋翼无人机的非线性动力学，我们建立简化的三轴姿态控制模型：

姿态通道方程：

$$\begin{aligned} I_x \ddot{\phi} &= \dot{\theta} \dot{\psi} (I_y - I_z) - C_1 \dot{\phi} + l U_2 + d_\phi \\ I_y \ddot{\theta} &= \dot{\phi} \dot{\psi} (I_z - I_x) - C_2 \dot{\theta} + l U_3 + d_\theta \\ I_z \ddot{\psi} &= \dot{\phi} \dot{\theta} (I_x - I_y) - C_3 \dot{\psi} + U_4 + d_\psi \end{aligned}$$

控制输入与电机转速关系：

$$\begin{bmatrix} U_2 \\ U_3 \\ U_4 \end{bmatrix} = \begin{bmatrix} -k_f l & 0 & k_f l & 0 \\ 0 & k_f l & 0 & -k_f l \\ k_m & -k_m & k_m & -k_m \end{bmatrix} \begin{bmatrix} \omega_1^2 \\ \omega_2^2 \\ \omega_3^2 \\ \omega_4^2 \end{bmatrix}$$

为便于Simulink实现，使用解耦的二阶系统近似每个通道：

滚转通道 (ϕ):

$$\ddot{\phi} + 0.8\dot{\phi} + 2\phi = U_2 + 0.1\dot{\theta}\dot{\psi} + d_\phi$$

俯仰通道 (θ):

$$\ddot{\theta} + 0.8\dot{\theta} + 2\theta = U_3 + 0.1\dot{\phi}\dot{\psi} + d_\theta$$

偏航通道 (ψ):

$$\ddot{\psi} + 1.0\dot{\psi} + 3\psi = U_4 + 0.05\dot{\phi}\dot{\theta} + d_\psi$$

传递函数形式：

$$G_{\phi}(s) = \frac{\Phi(s)}{U_2(s)} = \frac{1}{s^2 + 0.8s + 2}$$

$$G_{\theta}(s) = \frac{\Theta(s)}{U_3(s)} = \frac{1}{s^2 + 0.8s + 2}$$

$$G_{\psi}(s) = \frac{\Psi(s)}{U_4(s)} = \frac{1}{s^2 + 1.0s + 3}$$

这里，为体现四旋翼的非线性耦合特性，添加交叉耦合：

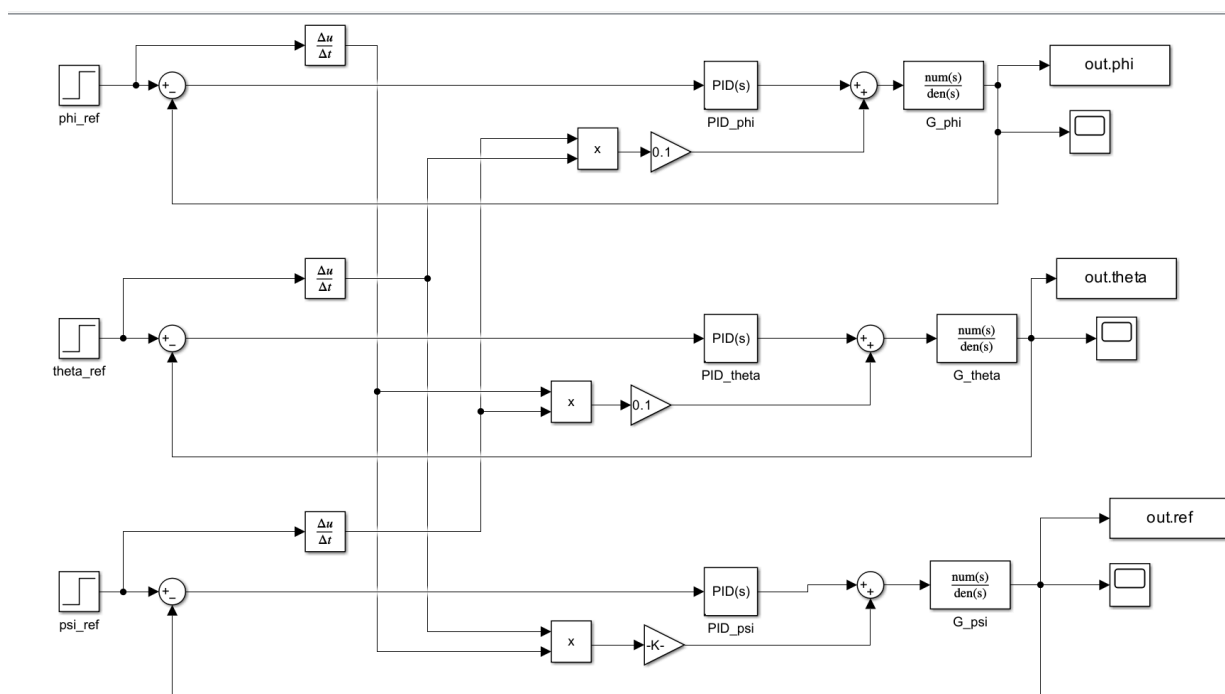
滚转耦合项： $0.1 \cdot \dot{\theta} \cdot \dot{\psi}$

俯仰耦合项： $0.1 \cdot \dot{\phi} \cdot \dot{\psi}$

偏航耦合项： $0.05 \cdot \dot{\phi} \cdot \dot{\theta}$

二、Simulink模型实现

1. 创建完整的PID三轴姿态控制模型



将这个simulink模型命名为pidsys。

2. PID调参

1. **比例增益 Kp**——决定系统响应的快慢和初步收敛能力。

增大 Kp 提升控制力度，加快输出接近期望值，但过大会降低相位裕度，引起振荡或超调。

2. **微分增益 Kd**——增加系统阻尼，抑制振荡。

Kd 对误差变化率敏感，相当于引入阻尼，改善二阶欠阻尼系统的峰值响应。Kd 过大可能导致响应迟滞或噪声放大，应结合传感器噪声滤波器调整。

3. 积分增益 Ki——消除稳态误差，实现精确跟踪。

Ki 对累积误差敏感，使系统在长期作用下输出逐步抵消稳态偏差。但Ki 太大可能产生积分饱和和低频振荡。

4. 调参顺序：Kp → Kd → Ki → 三轴联调

第一步：确定初始参数Kp。

设 $K_i = 0$ ， $K_d = 0$ ，只调整 K_p 。逐渐增大 K_p ，直到系统开始出现持续振荡（临界稳定状态）。记录此时的 K_p 值，记为 $K_{p_critical}$ （临界增益），并测量振荡周期 $T_{critical}$ 。

第二步：应用齐格勒-尼科尔斯（Ziegler-Nichols）经验公式。根据第一步测得的值计算整定参数。

控制器类型	Kp	Ki	Kd
P	$0.5 * K_{p_critical}$	0	0
PI	$0.45 * K_{p_critical}$	$0.54 * K_{p_critical} / T_{critical}$	0
PID	$0.6 * K_{p_critical}$	$1.2 * K_{p_critical} / T_{critical}$	$0.075 * K_{p_critical} * T_{critical}$

第三步：基于响应曲线精细调整

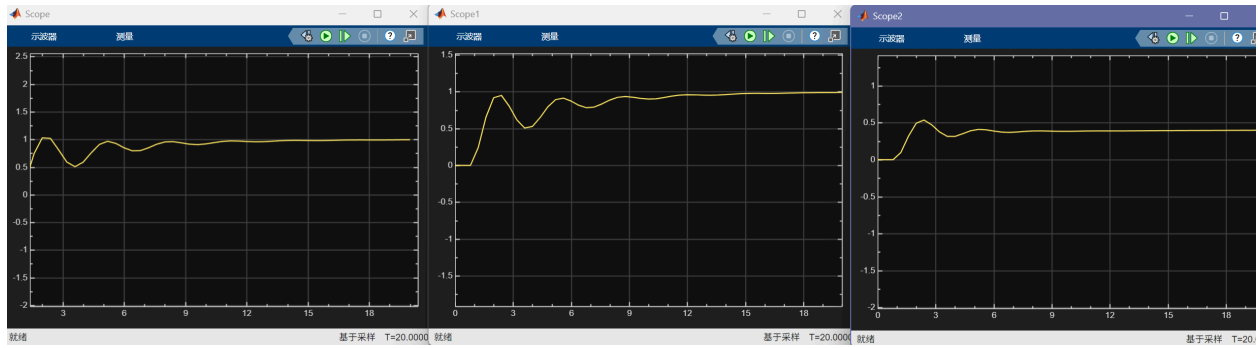
根据第二步的“粗参数”，观察阶跃响应曲线，按以下规律进行“微调”：

当前现象	问题根源	调整方向
响应太慢	系统“不够积极”	增大 Kp
超调过大、振荡	系统“过于激进”或阻尼不足	减小 Kp 或 增大 Kd
稳态误差持续存在	积分作用不足	增大 Ki
稳态误差反复过零	积分作用过强	减小 Ki
高频噪声被放大	微分对噪声敏感	减小 Kd

接下来，按照此规律进行了一系列的调参，并针对增加微分项后的大幅度震荡调整了滤波器参数。下面是确定的pid参数：

通道	Kp	Ki	Kd	滤波器系数	说明
滚转(φ)	2.5	1	0.05	1	响应最快，需较强微分抑制超调
俯仰(θ)	2	0.8	0.02	1	与滚转类似，耦合影响稍异
偏航(ψ)	1.8	0.01	0.01	1	转动惯量大，需较小比例增益

3. PID控制结果分析

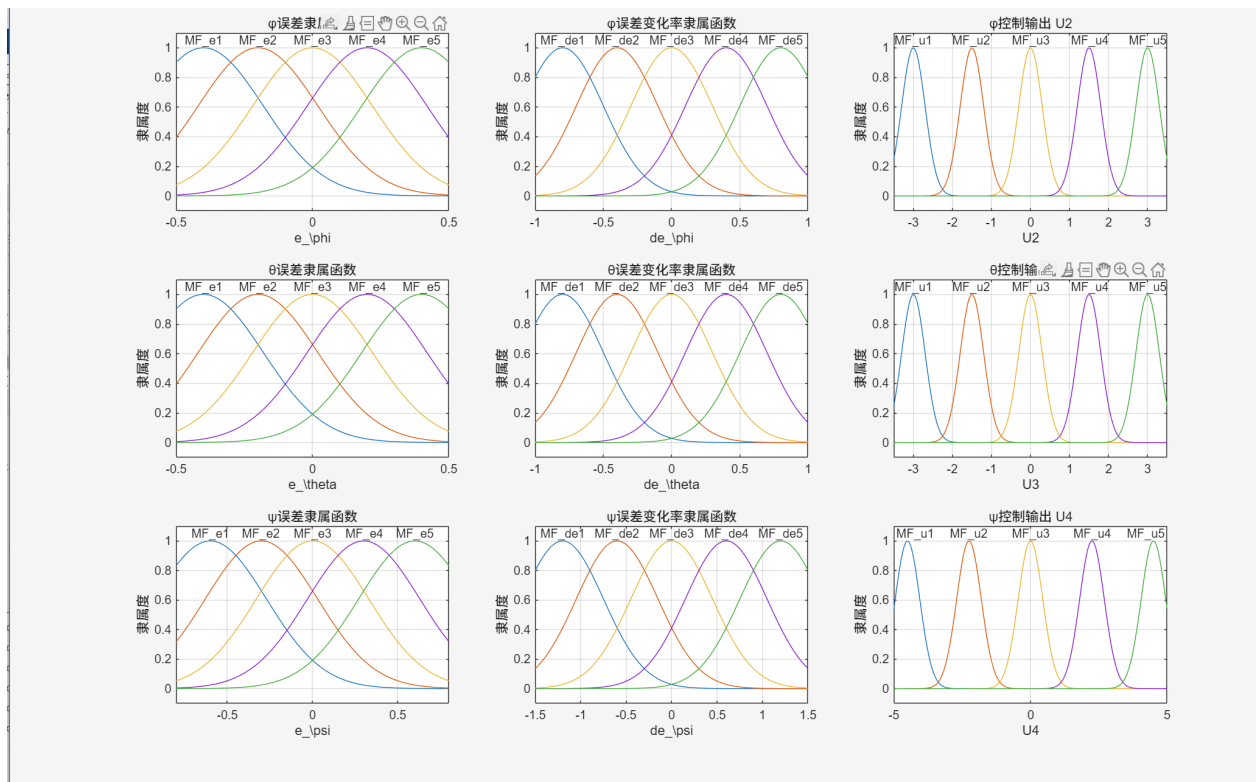


可以看出，我们的PID控制器虽然能够收敛稳定，但由于交叉耦合和扰动的存在，有响应时间长、有部分超调的劣势。此外，我在调参过程中发现，增加微分增益容易引起高频振荡，必须结合滤波器设计。接下来我使用了模糊控制、模糊自适应控制、BP神经网络这三种智能控制技术，希望能设计出效果更好的控制器。

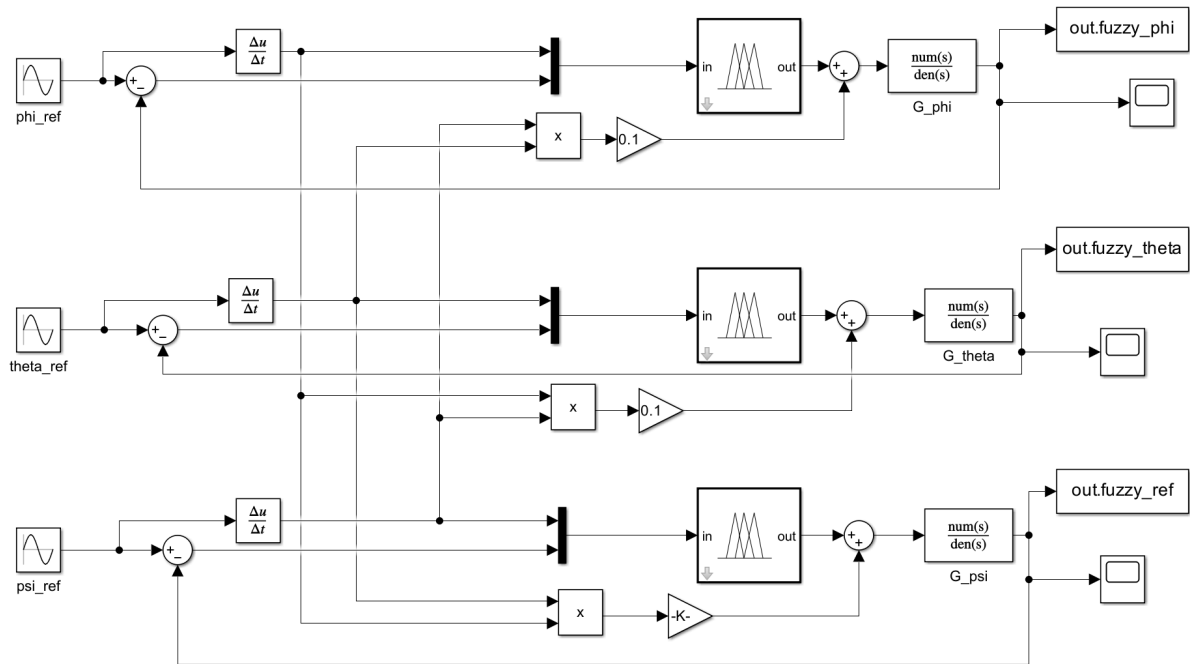
三、智能控制器对比实现

1.建立模糊控制：

1.1 普通模糊控制器



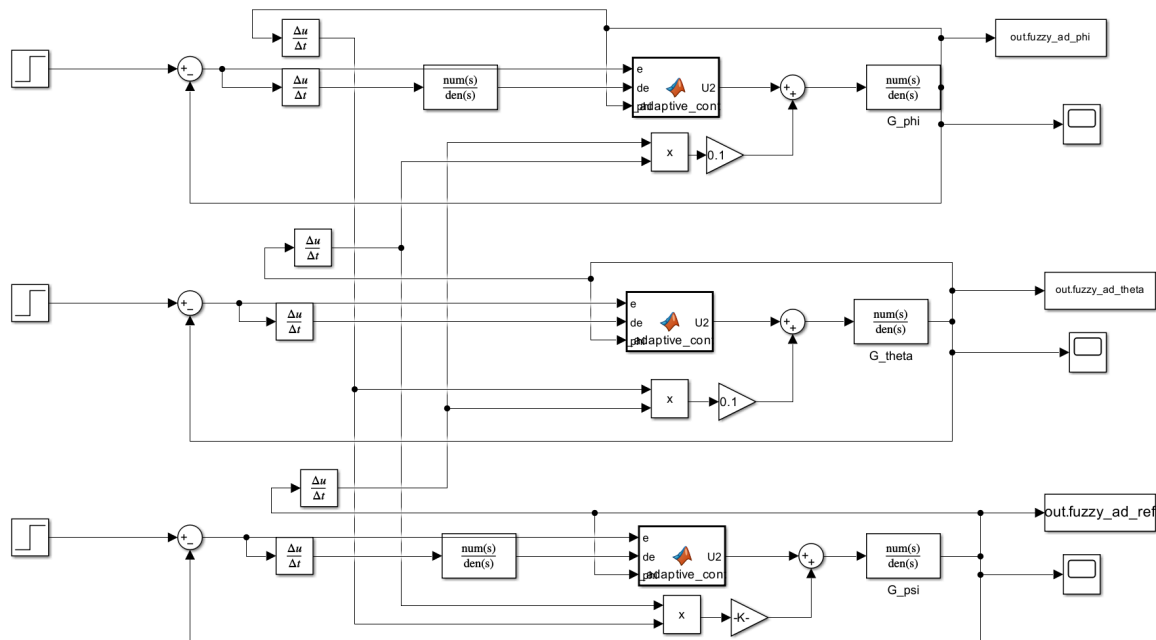
建立fuzzysys模型：



由于模糊控制的效果并不好，多次调整后仍然不能实现较好的收敛，可能是因为我模拟无人机系统时手动加入的扰动项会让模糊控制器束手无策。最后我没有采用模糊控制的方法，而是转向模糊自适应控制。

1.2. 模糊自适应控制器

使用matlab function硬编码模糊控制规则，并加入自适应机制实现。



初次仿真，phi跟踪产生震荡，psi和delta值非常小、几乎为0。为此，增加阻尼，调整自适应增益策略，使其更加平滑，并且在大误差时不要过度增加增益，完善规则如下：

- 规则1: 如果e负大且de负大，则U2正大（快速纠正）
- 规则2: 如果e负大且de负小，则U2正中
- 规则3: 如果e负小且de负大，则U2正中（增加阻尼）
- 规则4: 如果e负小且de负小，则U2正小
- 规则5: 如果e零且de负大，则U2正小（抑制振荡）
- 规则6: 如果e零且de零，则U2零
- 规则7: 如果e零且de正大，则U2负小（抑制振荡）
- 规则8: 如果e正小且de正小，则U2负小
- 规则9: 如果e正小且de正大，则U2负中（增加阻尼）
- 规则10: 如果e正大且de正大，则U2负大
- 规则11: 如果e正大且de正小，则U2负中
- 规则12: 如果实际角度phi负大，则U2正小（提供恢复力）
- 规则13: 如果实际角度phi正大，则U2负小（提供恢复力）

输出限幅调整为[-4,4]

修改后的phi输出如上图，可以看到震荡幅度减小，由于输出量不仅受控制量U的影响，还受交叉耦合项影响，这里再进行delta的修改：

- 减小输出限幅，从原来的 ± 4.5 减小到 ± 3.5 （或者更小，根据实际情况）。
- 减小自适应增益，特别是在大误差时的增益，避免过激的控制。

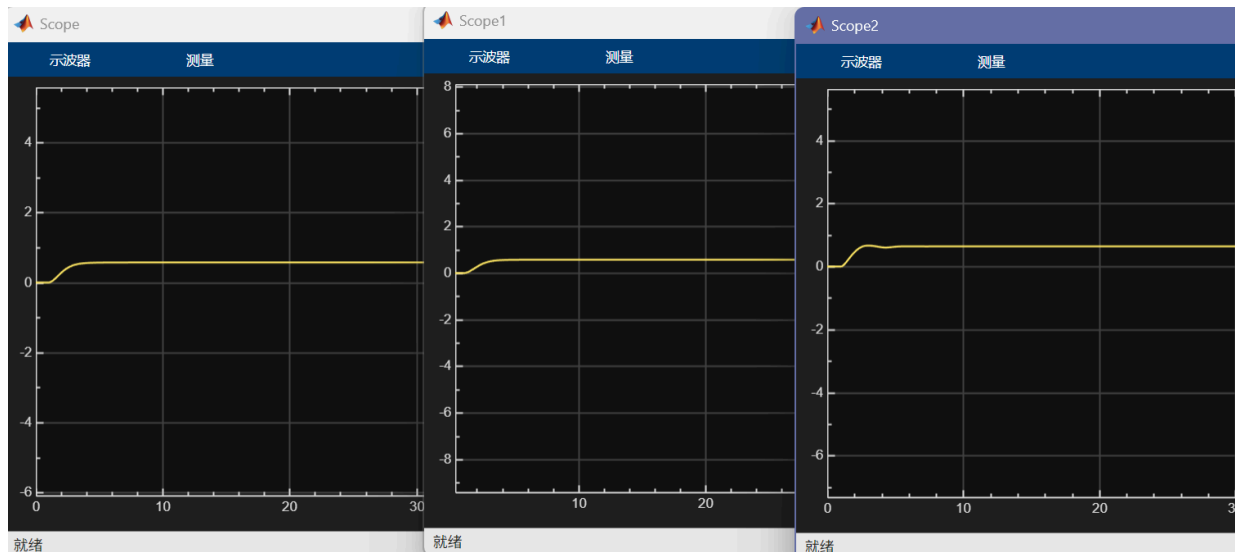
增加对误差变化率的考虑，增加阻尼。

考虑到耦合项的影响，当实际角度 θ 较大时，进一步减小控制量。

这时，发现输出震荡的震荡无法通过微调参数消除，分析主要原因在于：

自适应增益采用分段切换方式，导致控制参数时变不连续；同时模糊控制器本质等价于非线性 PD 控制，在欠阻尼二阶系统及耦合扰动作用下显著降低了系统相位裕度，从而引发极限环振荡。

我在 ϕ 和 ψ 的控制流程中对 de 加入一阶滤波，并对自适应律进行调整。滤波的效果是立竿见影的，震荡消失，控制效果明显提升：



最终控制器代码（以 ϕ 为例，其他部分进行了参数微调）：

```
function U2 = fuzzy_adaptive_controller(e, de, phi)
%  $\phi$  通道：连续自适应 + 高阻尼模糊控制器（稳定版）

%% ===== 输入限幅 =====
e = max(min(e, 0.5), -0.5);
de = max(min(de, 1.0), -1.0);
phi = max(min(phi, 0.8), -0.8);

%% ===== 隶属函数中心 =====
e_centers = [-0.4, -0.2, 0, 0.2, 0.4];
de_centers = [-0.8, -0.4, 0, 0.4, 0.8];
phi_centers = [-0.6, -0.3, 0, 0.3, 0.6];

U2_centers = [-3.0, -1.5, 0, 1.5, 3.0];
```

```

%% ===== 隶属度计算（更平滑） =====
e_mf = zeros(1,5);
de_mf = zeros(1,5);
phi_mf = zeros(1,5);

for i = 1:5
    e_mf(i) = gaussian_mf(e, e_centers(i), 0.22);
    de_mf(i) = gaussian_mf(de, de_centers(i), 0.30);
    phi_mf(i) = gaussian_mf(phi, phi_centers(i), 0.30);
end

%% ===== 模糊规则推理 =====
rule_strength = [];
rule_output = [];

% 规则 1: e NB & de NB → PB
rule_strength(end+1) = min(e_mf(1), de_mf(1));
rule_output(end+1) = U2_centers(5);

% 规则 2: e NB & de NS → PM
rule_strength(end+1) = min(e_mf(1), de_mf(2));
rule_output(end+1) = U2_centers(4);

% 规则 3: e ZE & de ZE → ZE
rule_strength(end+1) = min(e_mf(3), de_mf(3));
rule_output(end+1) = U2_centers(3);

% 规则 4: e PB & de PB → NB
rule_strength(end+1) = min(e_mf(5), de_mf(5));
rule_output(end+1) = U2_centers(1);

% 规则 5: e NS & de ZE → PS
rule_strength(end+1) = min(e_mf(2), de_mf(3));
rule_output(end+1) = U2_centers(4);

% 规则 6: e PS & de ZE → NS
rule_strength(end+1) = min(e_mf(4), de_mf(3));
rule_output(end+1) = U2_centers(2);

%% ===== 去模糊化 =====
if sum(rule_strength) > 0

```



```

        U2_base = sum(rule_strength .* rule_output) / sum(rule_strength);
    else
        U2_base = 0;
    end

    %% ===== 连续自适应增益（关键改动） =====
    % 连续、光滑、无跳变
    k_min = 0.4;
    k_max = 1.0;
    alpha = 4.0;

    adaptive_gain = k_min + (k_max - k_min) * tanh(alpha * abs(e));

    %% ===== 基于 de 的阻尼衰减（连续） =====
    beta = 2.5;
    damping_factor = 1 / (1 + beta * abs(de));

    %% ===== 最终控制输出 =====
    U2 = -1 * adaptive_gain * damping_factor * U2_base;

    %% ===== 输出限幅 =====
    U2 = max(min(U2, 3.5), -3.5);

    %% ===== 连续死区（避免抖振） =====
    if abs(e) < 0.03
        U2 = U2 * abs(e) / 0.03;
    end
end

%% ===== 高斯隶属函数 =====
function y = gaussian_mf(x, c, sigma)
    y = exp(-(x - c)^2 / (2 * sigma^2));
end

```

三、BP神经网络控制

3.1 神经网络设计

1. 神经网络结构

采用一个简单的前馈神经网络，包含一个输入层、一个或多个隐藏层和一个输出层

输入层：接受航向误差 e 和误差变化率 e_{dot} 。

隐藏层：神经网络对输入进行非线性变换。通常采用Sigmoid激活函数或ReLU激活函数。

输出层：输出舵角 δ ，控制信号。

2. 梯度下降法权值更新律

在BP神经网络中，梯度下降法用于最小化损失函数。使用均方误差（MSE）作为损失函数：

$$L = \frac{1}{2} \sum_i (y_i - \hat{y}_i)^2$$

其中， y_i 是真实输出， $\hat{y}_i$ 是神经网络的预测输出。

- 反向传播: 通过反向传播算法计算损失函数对各个权重的梯度。
- 梯度下降更新: 在每次迭代中，根据计算出的梯度更新网络中的权重。权值更新的公式为：

$$w = w - \eta \frac{\partial L}{\partial w}$$

其中：

w 是神经网络的权重。

η 是学习率，控制每次更新的步长。

$\frac{\partial L}{\partial w}$ 是损失函数对权重的梯度。

- 隐藏层到输出层的梯度计算

对于输出层的权重更新，我们计算损失函数对输出的梯度，通常使用链式法则：

$$\frac{\partial L}{\partial w_o} = \delta_o \cdot x_h$$

其中， δ_o 是输出层误差（即，目标输出和实际输出之间的误差）， x_h 是隐藏层的输出。

- 从输出层反向传播到隐藏层

通过链式法则，我们可以将误差从输出层反向传播到隐藏层，并且计算隐藏层的权重更新：

$$\frac{\partial L}{\partial w_h} = \delta_h \cdot x_i$$

其中， δ_h 是隐藏层误差， x_i 是输入数据。

3.2 神经网络训练

我们使用模糊自适应控制器运行时保存的输入与输出数据进行训练。

训练代码如下：

```
clc; close all;

%% ===== 1. 提取 timeseries 数据 =====
% phi 通道
e_phi = simout.e1.Data; % 误差  $\varphi$ 
```

```

de_phi = simout.de1.Data; % 误差导数  $\dot{\varphi}$ 
U_phi = simout.U2.Data; % 控制输出  $\varphi$ 

% delta 通道
e_delta = simout.e2.Data;
de_delta = simout.de2.Data;
U_delta = simout.U3.Data;

% psi 通道
e_psi = simout.e3.Data;
de_psi = simout.de3.Data;
U_psi = simout.U4.Data;

% 时间
t = simout.e1.Time; % 统一时间轴, 假设三个信号采样一致

%% ===== 2. 构造训练数据 =====
% phi 通道输入: [e, de], 输出: U
inputs_phi = [e_phi, de_phi]; % N×2
targets_phi = U_phi; % N×1

% delta 通道
inputs_delta = [e_delta, de_delta];
targets_delta = U_delta;

% psi 通道
inputs_psi = [e_psi, de_psi];
targets_psi = U_psi;

%% ===== 3. 归一化参数 =====
% phi
input_min_phi = min(inputs_phi, [], 1)';
input_max_phi = max(inputs_phi, [], 1)';
target_min_phi = min(targets_phi);
target_max_phi = max(targets_phi);

inputs_phi_norm = (inputs_phi - input_min_phi) ./ (input_max_phi - input_min_phi);
targets_phi_norm = (targets_phi - target_min_phi) / (target_max_phi - target_min_phi);

% delta
input_min_delta = min(inputs_delta, [], 1)';

```

```

input_max_delta = max(inputs_delta, [], 1)';
target_min_delta = min(targets_delta);
target_max_delta = max(targets_delta);

inputs_delta_norm = (inputs_delta - input_min_delta') ./ (input_max_delta' - input_min_delta');
targets_delta_norm = (targets_delta - target_min_delta) / (target_max_delta - target_min_delta);

% psi
input_min_psi = min(inputs_psi, [], 1)';
input_max_psi = max(inputs_psi, [], 1)';
target_min_psi = min(targets_psi);
target_max_psi = max(targets_psi);

inputs_psi_norm = (inputs_psi - input_min_psi') ./ (input_max_psi' - input_min_psi');
targets_psi_norm = (targets_psi - target_min_psi) / (target_max_psi - target_min_psi);

%% ===== 4. 转置为 Toolbox 格式 =====
% R×Q 输入，S×Q 输出
inputs_phi_train = inputs_phi_norm';
targets_phi_train = targets_phi_norm';

inputs_delta_train = inputs_delta_norm';
targets_delta_train = targets_delta_norm';

inputs_psi_train = inputs_psi_norm';
targets_psi_train = targets_psi_norm';

%% ===== 5. 创建并训练神经网络 =====
hidden_neurons = 10; % 可调

% phi
net_phi = feedforwardnet(hidden_neurons);
net_phi = configure(net_phi, inputs_phi_train, targets_phi_train);
[net_phi, tr_phi] = train(net_phi, inputs_phi_train, targets_phi_train);

% delta
net_delta = feedforwardnet(hidden_neurons);
net_delta = configure(net_delta, inputs_delta_train, targets_delta_train);
[net_delta, tr_delta] = train(net_delta, inputs_delta_train, targets_delta_train);

```

```

% psi
net_psi = feedforwardnet(hidden_neurons);
net_psi = configure(net_psi, inputs_psi_train, targets_psi_train);
[net_psi, tr_psi] = train(net_psi, inputs_psi_train, targets_psi_train);

%% ===== 6. 保存网络和归一化参数 =====
save('NN_Controller_Params.mat', ...
    'net_phi', 'input_min_phi', 'input_max_phi', 'target_min_phi', 'target_max_phi', ...
    'net_delta', 'input_min_delta', 'input_max_delta', 'target_min_delta', 'target_max_delta', ...
    'net_psi', 'input_min_psi', 'input_max_psi', 'target_min_psi', 'target_max_psi');

%% ===== 7. 训练结果评估 =====
% phi
outputs_phi = net_phi(inputs_phi_train)';
outputs_phi_denorm = outputs_phi * (target_max_phi - target_min_phi) + target_min_phi;

rmse_phi = sqrt(mean((targets_phi - outputs_phi_denorm).^2));
fprintf('phi RMSE: %.6f\n', rmse_phi);

% delta
outputs_delta = net_delta(inputs_delta_train)';
outputs_delta_denorm = outputs_delta * (target_max_delta - target_min_delta) + target_min_delta;
rmse_delta = sqrt(mean((targets_delta - outputs_delta_denorm).^2));
fprintf('delta RMSE: %.6f\n', rmse_delta);

% psi
outputs_psi = net_psi(inputs_psi_train)';
outputs_psi_denorm = outputs_psi * (target_max_psi - target_min_psi) + target_min_psi;
rmse_psi = sqrt(mean((targets_psi - outputs_psi_denorm).^2));
fprintf('psi RMSE: %.6f\n', rmse_psi);

%% ===== 8. 绘图 =====
figure;
subplot(3,1,1);
plot(t, targets_phi, 'b', 'LineWidth', 1.2); hold on;
plot(t, outputs_phi_denorm, 'r--', 'LineWidth', 1.2);

```

```

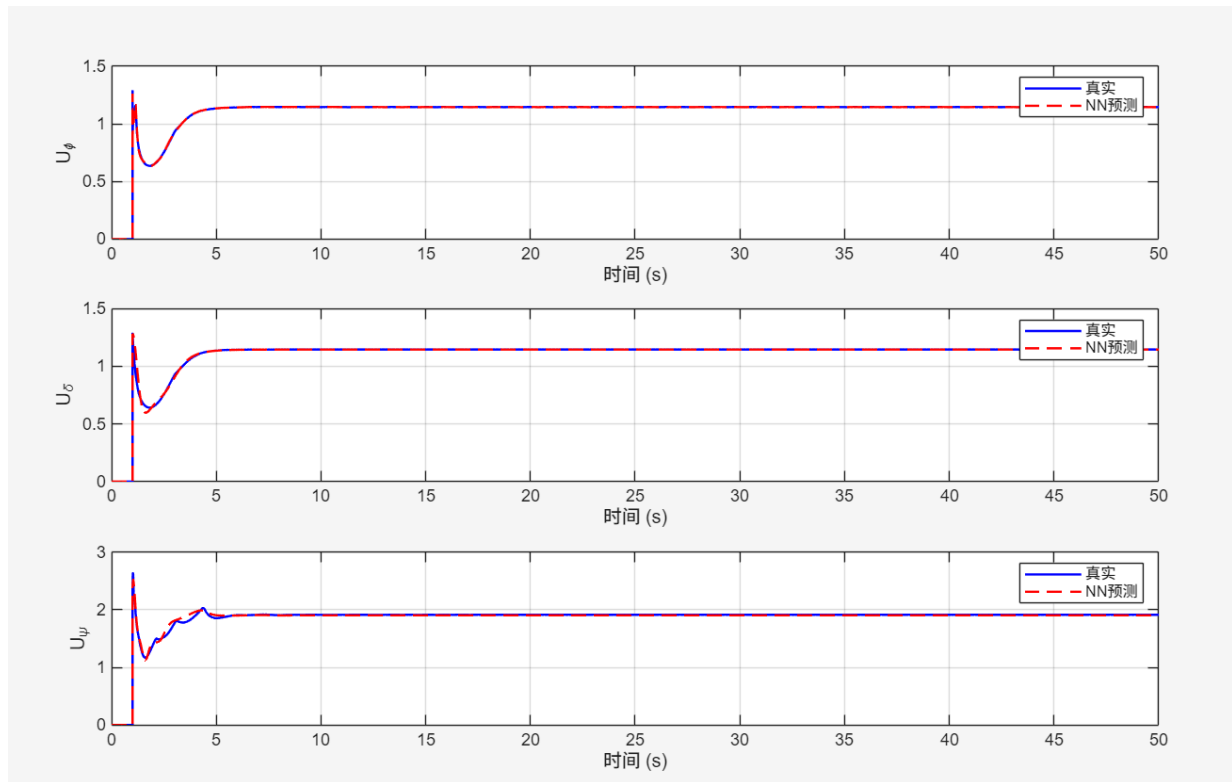
xlabel('时间 (s)'); ylabel('U_\phi');
legend('真实', 'NN预测'); grid on;

subplot(3,1,2);
plot(t, targets_delta, 'b', 'LineWidth', 1.2); hold on;
plot(t, outputs_delta_denorm, 'r--', 'LineWidth', 1.2);
xlabel('时间 (s)'); ylabel('U_\delta');
legend('真实', 'NN预测'); grid on;

subplot(3,1,3);
plot(t, targets_psi, 'b', 'LineWidth', 1.2); hold on;
plot(t, outputs_psi_denorm, 'r--', 'LineWidth', 1.2);
xlabel('时间 (s)'); ylabel('U_\psi');
legend('真实', 'NN预测'); grid on;

```

训练结果非常理想：



训练细节如下：

网络图

训练结果

训练结束: 满足验证条件 ✔

训练进度

单位	初始值	停止值	目标值
轮	0	12	1000
历时	-	00:00:00	-
性能	0.123	2.52e-05	0
梯度	0.416	0.00405	1e-07
Mu	0.001	1e-05	1e+10
验证检查	0	6	6

训练算法

数据划分: 随机 dividerand

训练: 莱文贝格-马夸特 trainlm

性能: 均方误差 mse

计算: MEX

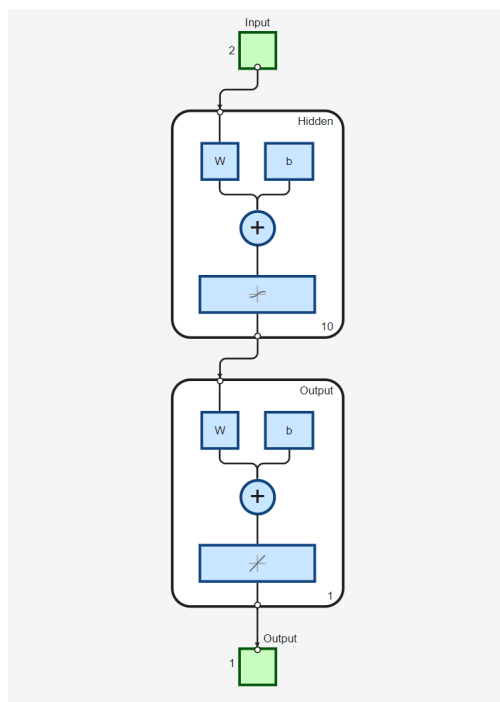
训练图

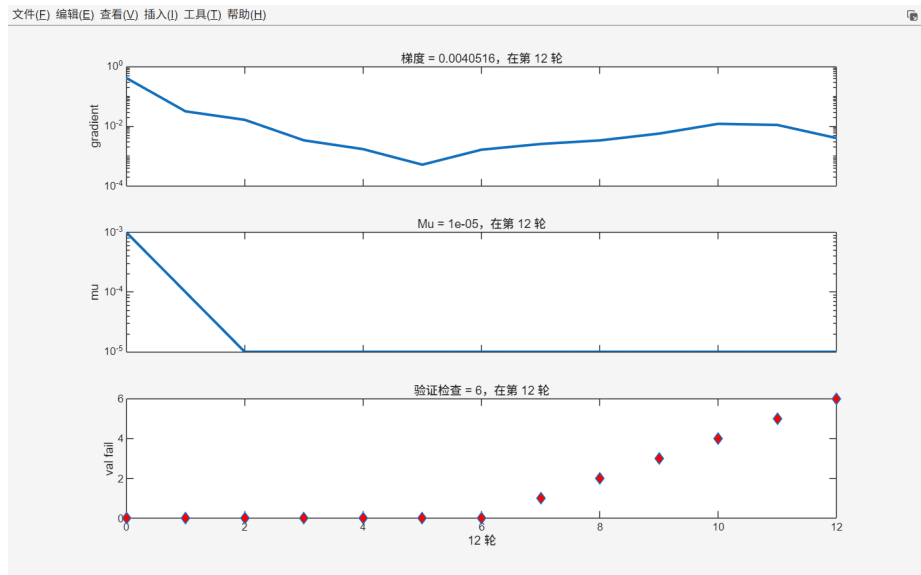
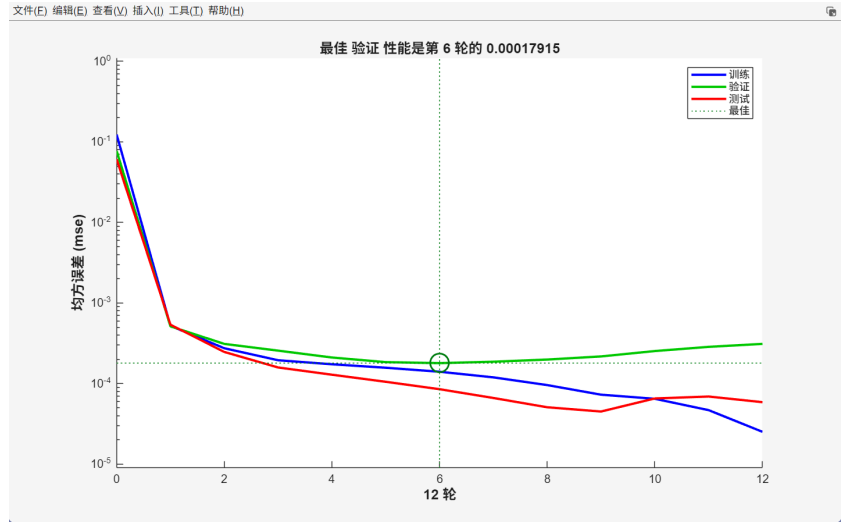
性能

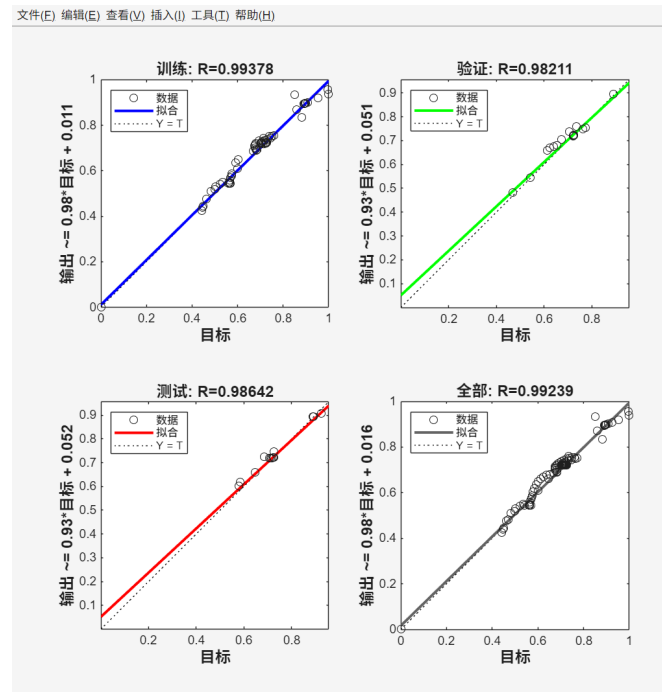
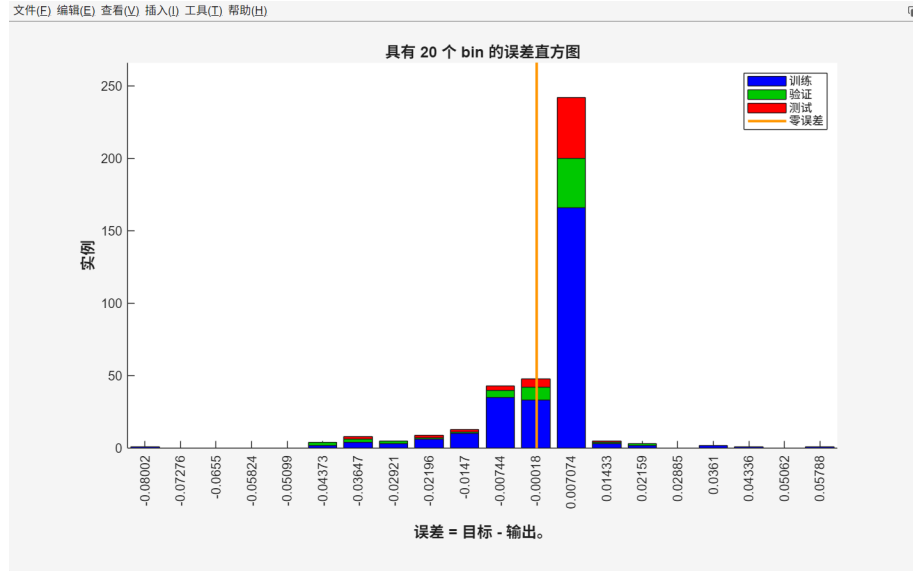
训练状态

误差直方图

回归







3.2 神经网络控制器搭建与仿真

在 Simulink 中使用 **MATLAB Function** 模块实现神经网络的前向计算，在 **MATLAB Function** 模块中编写以下代码：

```
function [U_phi, U_delta, U_psi] = NN_Controller(e_phi, de_phi, e_delta, de_delta, e_psi, de_psi)
%#codegen
% 三通道神经网络控制器
```

```

% 输出显式初始化
U_phi = 0.0;
U_delta = 0.0;
U_psi = 0.0;
% 使用外部函数
coder.extrinsic('nn_predict_channel');
% 调用三通道神经网络
U_phi = double(nn_predict_channel(e_phi, de_phi, 'phi'));
U_delta = double(nn_predict_channel(e_delta, de_delta, 'delta'));
U_psi = double(nn_predict_channel(e_psi, de_psi, 'psi'));

end

```

同时编写nn_predict_channel.m函数：

```

function y = nn_predict_channel(e, de, channel)
% 单通道 NN 前向仿真 (feedforwardnet + persistent)
persistent net_phi net_delta net_psi
persistent input_min_phi input_max_phi target_min_phi target_max_phi
persistent input_min_delta input_max_delta target_min_delta target_max_delta
persistent input_min_psi input_max_psi target_min_psi target_max_psi

if isempty(net_phi)
    data = load('NN_Controller_Params.mat', ...
        'net_phi','input_min_phi','input_max_phi','target_min_phi','target_max_phi', ...
        'net_delta','input_min_delta','input_max_delta','target_min_delta','target_max_delta', ...
        'net_psi','input_min_psi','input_max_psi','target_min_psi','target_max_psi');

    net_phi = data.net_phi;
    input_min_phi = data.input_min_phi;
    input_max_phi = data.input_max_phi;
    target_min_phi = data.target_min_phi;
    target_max_phi = data.target_max_phi;

    net_delta = data.net_delta;
    input_min_delta = data.input_min_delta;
    input_max_delta = data.input_max_delta;
    target_min_delta = data.target_min_delta;
    target_max_delta = data.target_max_delta;

    net_psi = data.net_psi;

```

```

input_min_psi = data.input_min_psi;
input_max_psi = data.input_max_psi;
target_min_psi = data.target_min_psi;
target_max_psi = data.target_max_psi;
end

% 选择通道
switch channel
case 'phi'
    net = net_phi;
    input_min = input_min_phi;
    input_max = input_max_phi;
    target_min = target_min_phi;
    target_max = target_max_phi;
case 'delta'
    net = net_delta;
    input_min = input_min_delta;
    input_max = input_max_delta;
    target_min = target_min_delta;
    target_max = target_max_delta;
case 'psi'
    net = net_psi;
    input_min = input_min_psi;
    input_max = input_max_psi;
    target_min = target_min_psi;
    target_max = target_max_psi;
otherwise
    error('未知通道名称');
end

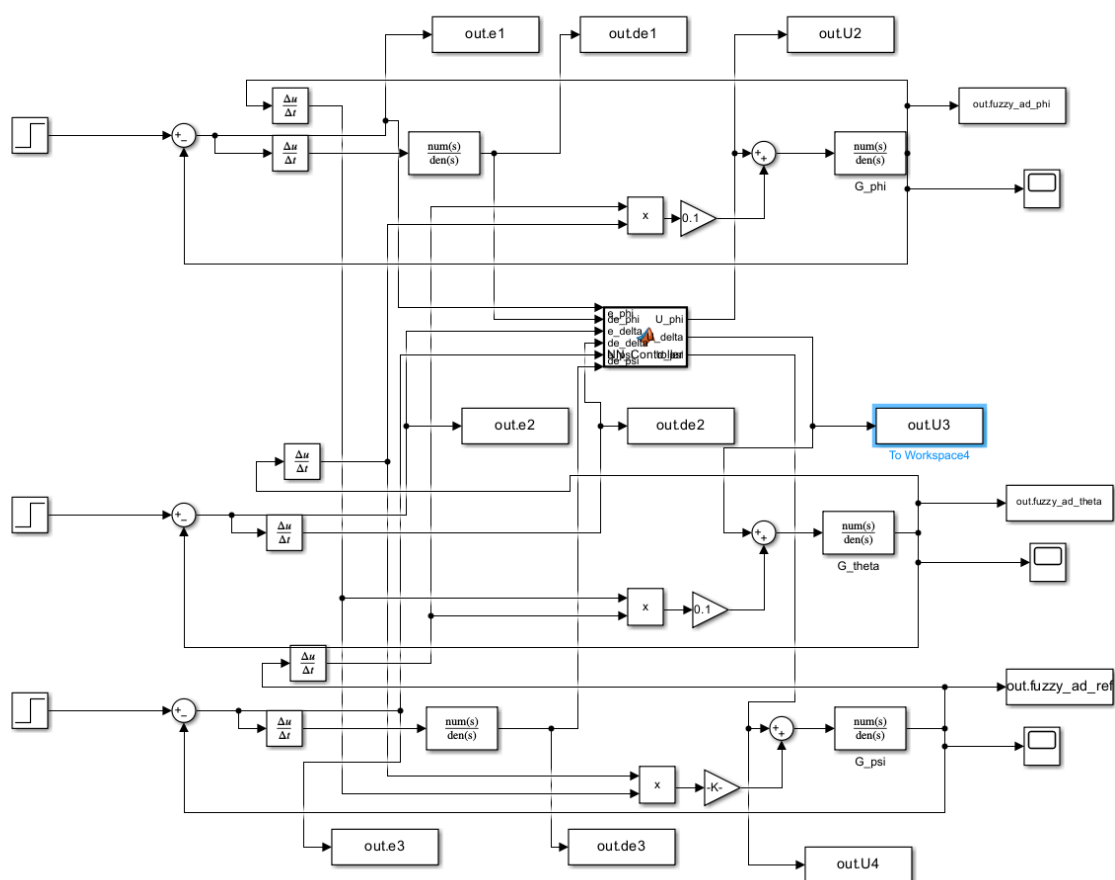
% 构造输入并归一化
u = [e; de];
u_norm = (u - input_min(:)) ./ (input_max(:) - input_min(:));

% 前向仿真
y_norm = sim(net, u_norm); % 1×1

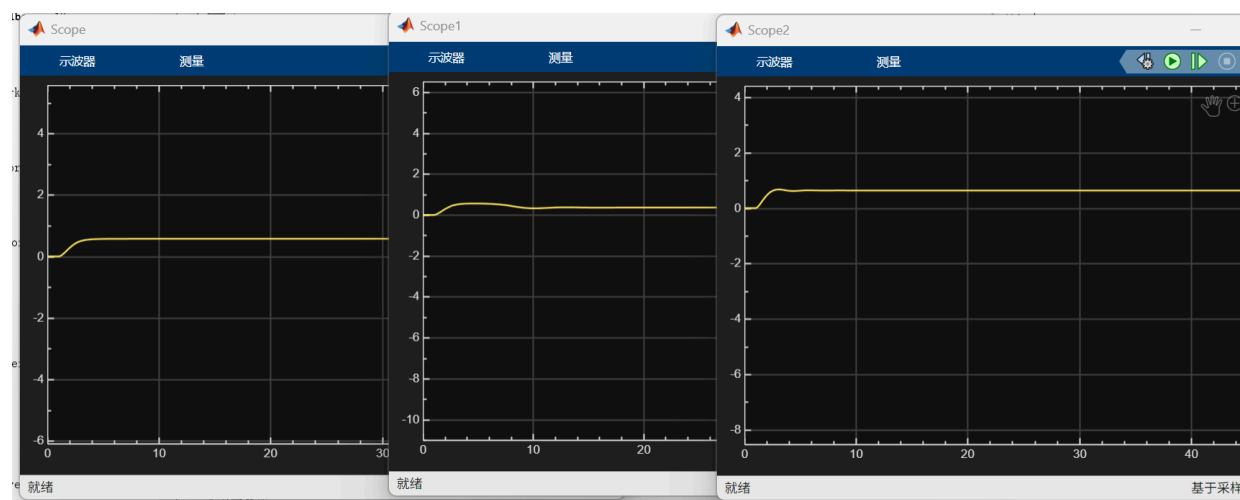
% 反归一化
y = y_norm * (target_max - target_min) + target_min;
end

```

进行一系列调试后，这个含有神经网络控制器的仿真系统能够顺利运行：



可以看到，神经网络控制器的控制效果也非常理想



四、对比与总结

1. 三轴姿态响应对比

- PID：快速，存在超调，耦合扰动影响明显
- 模糊自适应：没有超调，几乎没有震荡，但大误差下响应略慢
- 神经网络：没有超调，有一点震荡，对模糊自适应拟合良好

2. 结论

1. 传统PID控制器简单稳定、易于实现，但面对非线性和耦合扰动表现欠佳，无论怎么调试都会有一些震荡才能收敛。
2. 模糊自适应控制通过规则库与连续增益策略，提高了系统的鲁棒性，但它对输入项的平滑程度有要求，需要适当加入滤波环节来辅助，还需要精细的调参，这也是我本次作业耗费最多时间的一部分。
3. 神经网络控制器通过学习控制规律，实现最优姿态控制，它能够较好地补偿耦合和扰动，对于我选择的这个较为复杂的“四旋翼无人机”系统，也是一个很好的选择方案。

综合来看，本次大作业实现了从理论建模 → PID设计 → 模糊自适应 → 神经网络控制的完整尝试，探索了非线性系统控制的挑战及解决方法。

3. 心得体会

《智能控制技术》是我第一次接触并使用simulink工具的课程。在完成四次小作业和大作业的过程中，我学到了很多实用的控制方法，获得了满满的simulink调试经验。虽然有的时候控制的结果并不尽如人意，但我在一次次实践与调试的过程中，获得了对于“控制”更深入的理解，看着自己在《自动控制原理》课程中学到的书本知识展现在我的眼前，我也很有成就感。在完成本次大作业的过程中，我也遇到了很多大大小小的问题，经过了反复的思考与尝试，才得到了令人满意的控制效果。感谢老师和助教在这个冬学期的对我们的教导和帮助！